



TENSORORBIT · EDA BOOTCAMP

EXPLORATORY DATA ANALYSIS BOOTCAMP

DAY 1 — FOUNDATIONS OF DATA SCIENCE, PYTHON & PANDAS

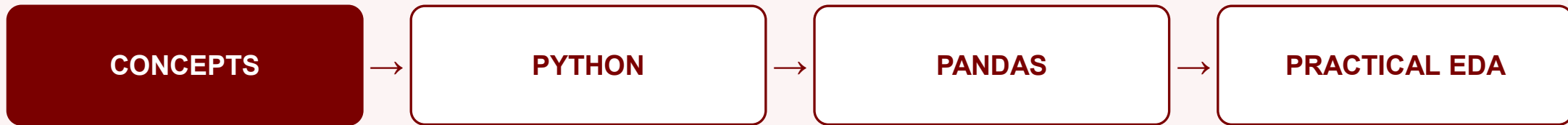
From raw data to meaningful insights.

INSTRUCTOR

Abdou Salam Sisawo

21 – 29 AUG · 4 SESSIONS

How We'll Get There



0–10 min

Welcome & bootcamp overview

10–25 min

Data Science + Data Roles

25–40 min

EDA concepts + workflow

40–70 min

Python fundamentals

70–80 min

Short break / questions

80–120+ min

Pandas + guided dataset exploration

Structure allows the instructor to naturally extend past 2 hours if participants have questions.

01

DATA SCIENCE

Where questions meet data

CONCEPT

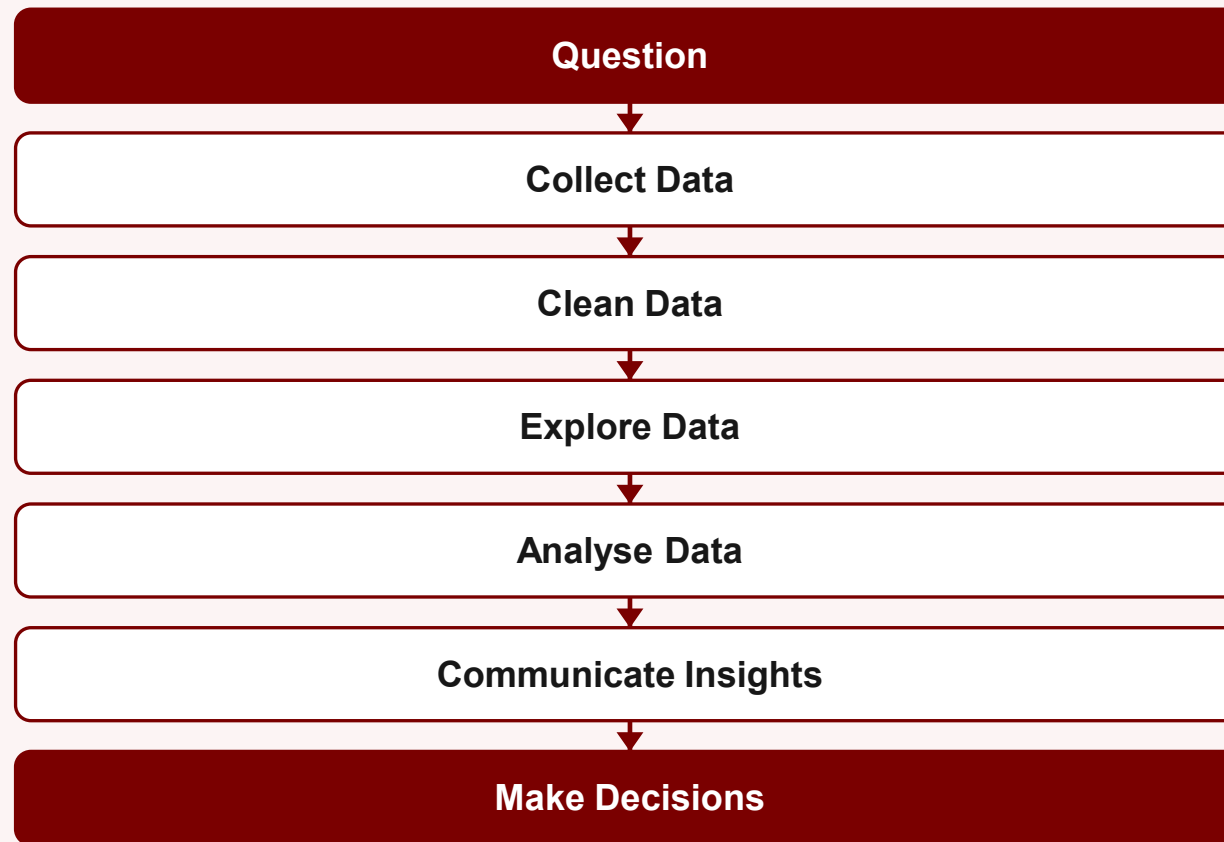
What Is Data Science?

Data Science is the process of using data, statistics, programming, and domain knowledge to answer questions, discover patterns, make predictions, and support decisions.

- It is broader than Machine Learning — ML is one tool inside Data Science, not the whole field
- It blends curiosity, code, and business/domain context
- It always starts with a question — the data comes second

DIAGRAM

The Data Science Workflow



EDA lives right in the middle of this workflow — Explore & Analyse.

02

WHO WORKS WITH DATA?

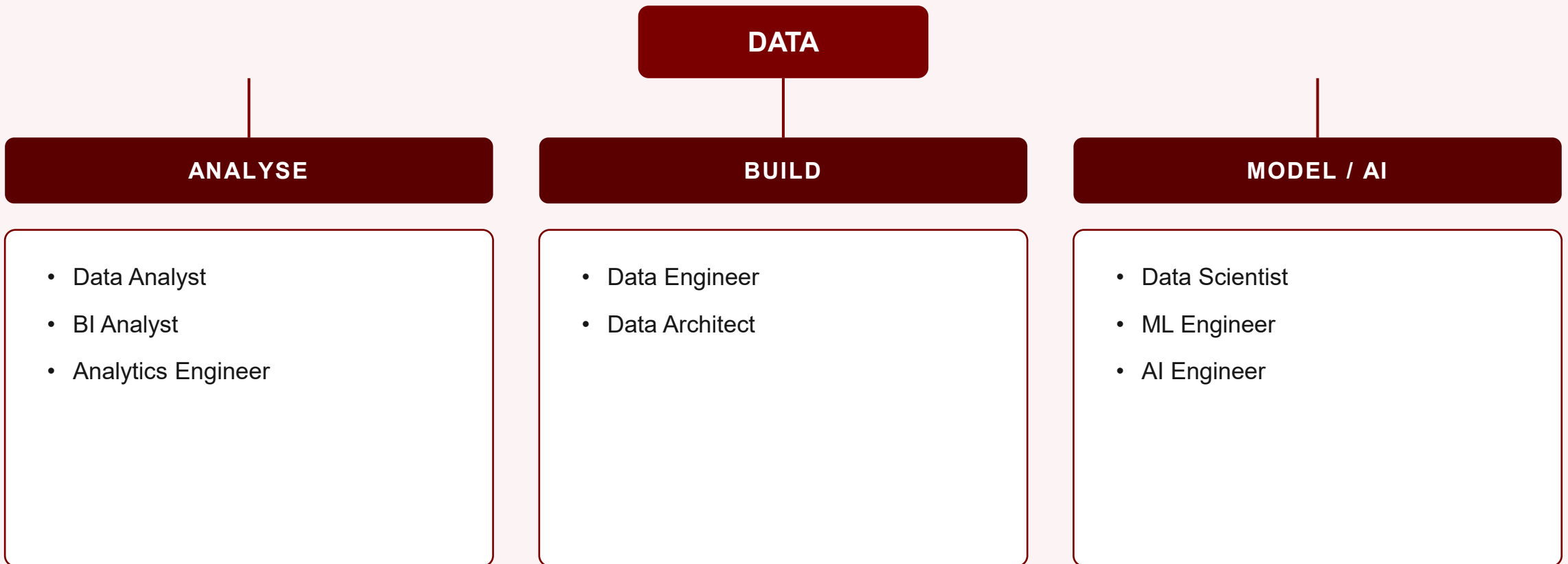
Same field, many different jobs

Data Roles at a Glance

Role	Main Question	What They Do	Common Tools
Data Analyst	“What happened?”	Reports on past & current trends	Excel, SQL, Power BI
Data Scientist	“Why did it happen? What's next?”	Builds models & finds patterns	Python, R, ML libraries
Data Engineer	“How do we get reliable data?”	Builds pipelines & infrastructure	SQL, Spark, Airflow
ML Engineer	“How do we put ML into production?”	Deploys & scales models	Python, Docker, Cloud
BI Analyst / Developer	“How can the business understand its data?”	Builds dashboards & reports	Power BI, Tableau, SQL
Analytics Engineer	“How do we turn raw data into usable data?”	Models & shapes data for analysis	dbt, SQL, Python
Data Architect	“How should our data systems be designed?”	Designs data structure & strategy	SQL, Cloud platforms
AI Engineer	“How do we build apps using AI models?”	Integrates AI/LLMs into products	Python, APIs, LLM tools

DIAGRAM

How These Roles Group Together



These roles overlap — job titles vary from company to company.

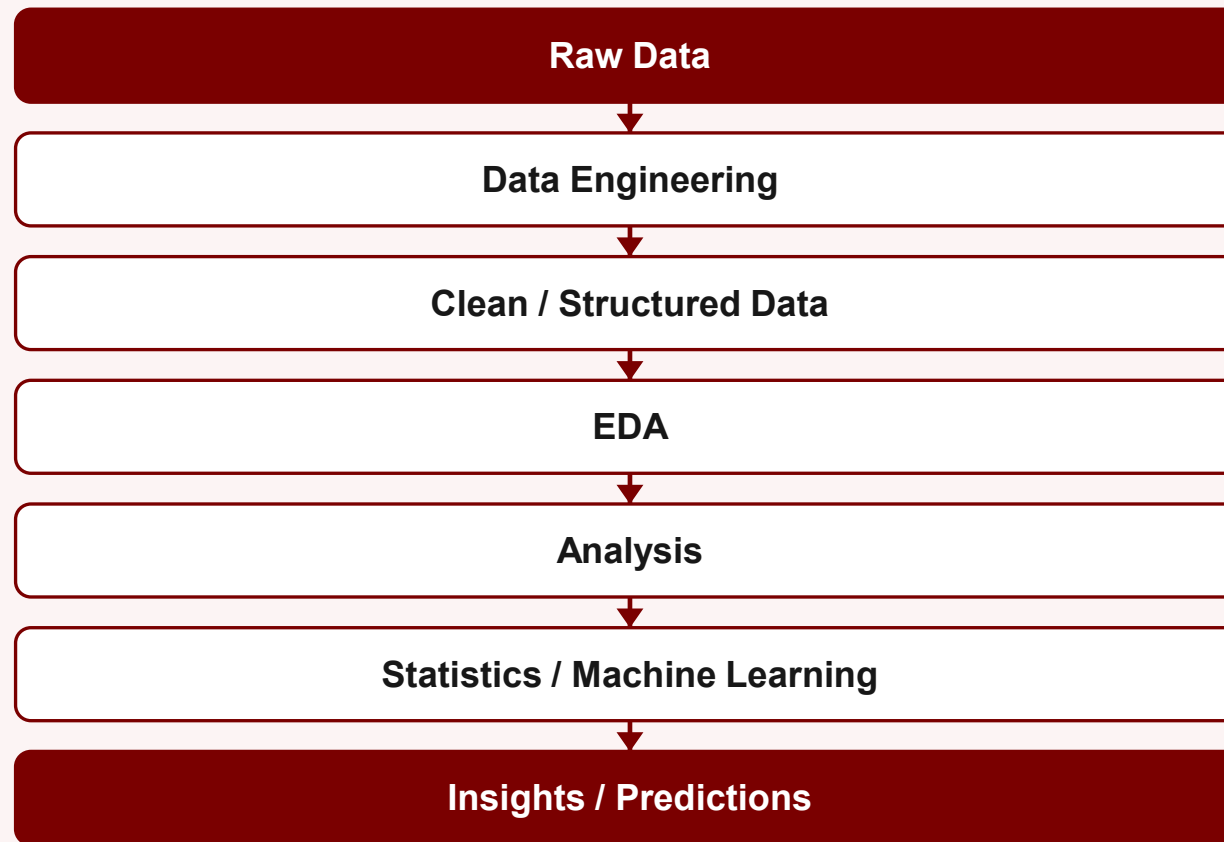
03

EXPLORATORY DATA ANALYSIS

Getting to know your data before you trust it

DIAGRAM

Where Does EDA Fit?



Before building a model, how do we know our data makes sense?

CONCEPT

What Is EDA?

EDA = Exploratory Data Analysis. Looking closely at a dataset before you analyse or model it.

- Understand the data
- Find missing values
- Find duplicates
- Detect unusual values
- Understand distributions
- Identify relationships
- Decide what to investigate next

THINK BEFORE YOU CODE

What could go wrong if we analyse data without checking it first?

Duplicate rows can quietly inflate your averages

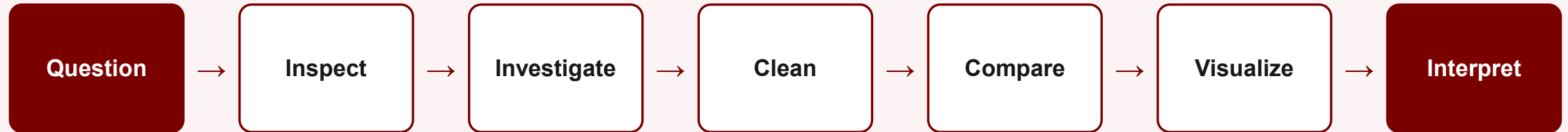
A missing 'Salary' can silently break a calculation

Text stored where numbers belong can crash your analysis

Discuss with the person next to you, then we'll walk through it together.

DIAGRAM

EDA Is a Conversation With Your Data



“EDA is not just running commands.”

Questions this raises:

How many employees are there?

Are there missing ages?

Any duplicate employees?

What is the salary distribution?

Highest average salary by department?

Do remote employees differ by department?

04

PYTHON FUNDAMENTALS

Only what you need to speak Pandas fluently

CONCEPT

We're Not Learning All of Python

- Goal: just enough Python to read and write Pandas code confidently
- We will skip: classes, decorators, recursion, generators
- We will cover: variables, data types, operators, strings, lists, dictionaries, conditions, loops, functions, imports

Every concept in this section reappears inside Pandas in the next section — nothing here is wasted.

CODE

Variables & Data Types

“How do we store a value in Python?”

```
name = "Alice"  
age = 25  
salary = 50000  
remote = True
```

- A variable is just a name we give to a value, so we can reuse it
- str = text, int = whole number, float = decimal, bool = True/False, None = nothing
- Python figures out the type automatically — no need to declare it up front

THINK BEFORE YOU CODE

**“25” and 25 look the same to us.
Are they the same to Python?**

Quotes = text (str) · No quotes = number (int)

This exact mix-up is why a 'Salary' column full of text can't be averaged in Pandas later

Discuss with the person next to you, then we'll walk through it together.

CODE

Operators

```
age > 30  
salary >= 50000  
age > 30 and remote == True
```

- Arithmetic: + - * / // % **
- Comparison: == != > < >= <=
- Logical: and or not

Remember this last line — we'll write this exact logic to filter an entire spreadsheet of employees very soon.

CODE

Strings

```
name = "Abdou"

name[0]          # indexing
name[0:3]        # slicing

name.lower()
name.strip()
f"Employee: {name}" # f-string
```

- Indexing grabs one character; slicing grabs a range
- .lower(), .strip(), .replace() clean up messy text
- f-strings let us combine text and variables in one readable line

Messy real-world names, emails, and job titles need exactly these tools — we'll use .str versions of these in Pandas.

CODE

Lists

```
employees = ["Alice", "Bob", "Charlie"]  
  
employees[0]  
employees[0:2]  
employees.append("David")  
  
for employee in employees:  
    print(employee)
```

- A list holds many values, in order, inside one variable
- `.append()` adds a new item to the end
- A for loop repeats an action once for every item in the list

A list is like one column of values sitting in order — that's exactly what a table column is.

CODE

Dictionaries

```
employee = {  
    "name": "Alice",  
    "age": 25,  
    "department": "HR"  
}  
  
employee["name"]
```

- A dictionary stores key → value pairs
- We look up a value using its key, not its position

A DataFrame row behaves a lot like a dictionary — the column name is the key.

Conditions

Python condition



Pandas filter

```
if salary > 50000:  
    print("High salary")  
else:  
    print("Other")
```

- if / elif / else let our code make decisions
- The exact same 'true or false' logic will filter an entire DataFrame later — just applied to a whole column at once instead of one variable

Loops

```
for employee in employees:  
    print(employee)  
  
for i in range(5):  
    print(i)
```

- A for loop repeats an action for every item in a sequence
- `range(5)` produces the numbers 0, 1, 2, 3, 4
- Pandas will do this kind of repetition for us automatically — but it helps to see it manually once

Functions

```
def calculate_total_salary(salary, bonus):  
    return salary + bonus  
  
calculate_total_salary(50000, 2000)
```

- `def` starts a function definition; the values in parentheses are parameters
- `return` sends a result back out of the function
- Functions let us reuse logic instead of retyping it

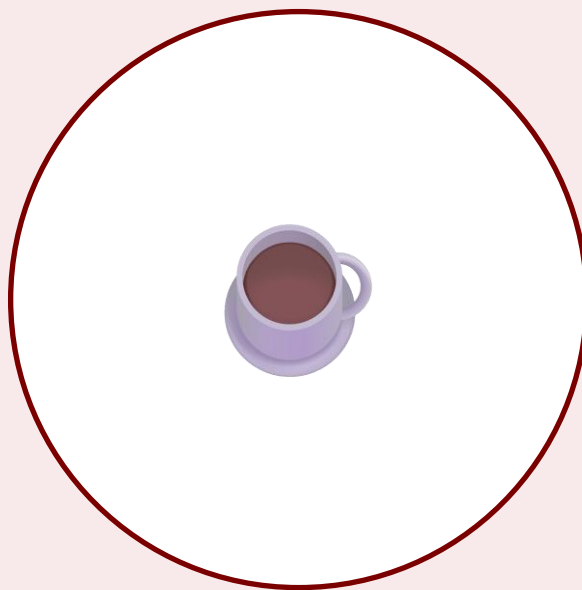
CODE

Imports — The Bridge to Pandas

```
import pandas as pd  
import numpy as np
```

- import brings an external library's tools into our file
- as pd / as np creates a short alias — we'll type pd. hundreds of times today
- pd and np are just nicknames, not magic keywords

This is the exact line every notebook you write from now on will start with.



Quick Break

Stretch, refill your coffee, and ask a question.

Which Python idea from the last section is still fuzzy?

Write down one question about your own data — we'll revisit it in the guided challenge.

05

MEET PANDAS

Python's tool for tables

CONCEPT

What Is Pandas?

Pandas is a Python library built for working with structured, tabular data — rows and columns, just like Excel.

- Built on top of the exact Python we just learned
- Two core building blocks: Series and DataFrame
- Nearly every EDA task from here on starts with Pandas

CODE

Series — One Column of Data

```
pd.Series([10, 20, 30])
```

- A Series is a single labelled column of values
- Think of it as a list, but with an index attached

CODE

DataFrame — A Full Table

```
pd.DataFrame({  
    "name": ["Alice", "Bob"],  
    "age": [25, 30]  
})
```

- Rows = records (one employee per row)
- Columns = variables (name, age, department...)
- Notice the shape: a dictionary of lists becomes a table

This is the exact dictionary pattern from a few slides ago — now holding entire columns instead of single values.

06

WORKING WITH MESSY DATA

Meet the Messy Employee Dataset

DATASET

The Messy Employee Dataset

1,000+

ROWS

13

COLUMNS

Source: Kaggle — “Messy Employee Dataset” (desolution01)

kaggle.com/datasets/desolution01/messy-employee-dataset

13 Columns

Employee_ID

First_Name

Last_Name

Name

Age

Department_Region

Status

Join_Date

Salary

Email

Phone

Performance_Score

Remote_Work

Known data-quality issues:

Duplicates · Missing values · Inconsistent date formats · Invalid / non-numeric salary values · Compound columns (Department_Region)

CONCEPT

Why Did We Choose a Messy Dataset?

Real-world data is rarely clean. The goal isn't 'can you run Pandas?' — it's 'can you look at data and figure out what needs to happen next?'

CODE

Loading the Data

```
import pandas as pd

df = pd.read_csv("Messy_Employee_dataset.csv")
```

- `pd.read_excel("file.xlsx")` works the same way for Excel files
- `df` is now our entire spreadsheet, living in memory as a DataFrame

CODE

First Look at the Data — Shape & Structure

“How big is my dataset?”



```
df.shape
```

“What variables do I have?”



```
df.columns
```

“What do the first rows look like?”



```
df.head()
```

“What about the last rows?”



```
df.tail()
```

Run each command and actually look at the output before moving to the next one — don't rush this first inspection.

CODE

First Look at the Data — Types & Summary

“What are the data types & missing values?”



```
df.info()
```

“What do the numeric columns look like?”



```
df.describe()
```

df.info() is often the single most useful first command on any new dataset — it tells you row counts, types, and missing values in one shot.

Selecting Data

```
df["Salary"]  
df[["Name", "Salary", "Department_Region"]]  
  
df.loc[0, "Salary"]  
df.iloc[0, 4]
```

- `df["col"]` selects one column; `df[...]` selects several
- `.loc[]` selects by label (row/column name)
- `.iloc[]` selects by position (row/column number)

CODE

Filtering Data

```
df[df["Age"] > 40]
df[df["Status"] == "Active"]

df[(df["Age"] > 30) & (df["Remote_Work"] == True)]
```

Remember: *age > 30 and remote == True* — same logic, now applied to an entire column instead of one variable.

Sorting Data

“Who are the highest-paid employees?”

```
df.sort_values("Salary")  
df.sort_values("Salary", ascending=False)  
  
df.nlargest(10, "Salary")
```

- ascending=False flips the order to highest-first
- df.nlargest(10, "Salary") jumps straight to the top 10

CODE

Descriptive Statistics

```
df["Salary"].mean()  
df["Salary"].median()  
df["Status"].value_counts()
```

- What is the average employee age?
- What is the median salary?
- How many employees are Active vs Inactive?
- What is the highest salary in the dataset?

Missing Values

```
df.isnull().sum()

df["Age"].fillna(df["Age"].median())
df.dropna()
```

- Missing data is not automatically “bad” — first we investigate
- Ask: How much is missing? Which columns? Why might it be missing? What should we do?
- `fillna()` replaces gaps; `dropna()` removes rows entirely — the right choice depends on the question

Duplicates

```
df.duplicated().sum()  
df.drop_duplicates()
```

- duplicated() flags rows that repeat an earlier row
- If the same employee appears twice, counts and averages become misleading

CODE

Data Types & Conversion

```
df.dtypes
```

```
df["Salary"] = pd.to_numeric(df["Salary"], errors="coerce")  
df["Join_Date"] = pd.to_datetime(df["Join_Date"], errors="coerce")
```

- A column that looks numeric to a human may secretly be stored as text
- errors="coerce" turns anything unreadable into a missing value instead of crashing

This is exactly the “25” vs 25 problem from the Python section — now showing up inside a real column.

CODE

Creating & Modifying Columns

```
df["Annual_Salary"] = df["Salary"] * 12  
df.rename(columns={"Name": "Full_Name"})
```

- New columns can be built from existing ones — this is called feature creation
- `rename()` relabels columns without touching the underlying data

CODE

String Operations

```
df["Name"].str.strip()  
df["Name"].str.lower()  
df["Email"].str.contains("@gmail")
```

- The `.str` accessor applies string methods to an entire column at once
- Inconsistent capitalization and stray whitespace are everywhere in messy data

CODE

Working With Dates

“What year did most employees join?”

```
df["Join_Date"] = pd.to_datetime(df["Join_Date"], errors="coerce")

df["Join_Date"].dt.year
df["Join_Date"].dt.month
df["Join_Date"].dt.day_name()
```

- The .dt accessor unlocks year, month, day, and weekday from a datetime column

GroupBy & Aggregation

Split



Apply



Combine

```
df.groupby("Department_Region")["Salary"].mean()

df.groupby("Status")["Salary"].agg(
    ["count", "mean", "min", "max"]
)
```

- Which department/region has the highest average salary?
- How many employees are in each status?
- What is the average salary by performance score?

Merge & Concat

concat — stack datasets



Combined DataFrame

merge — join on a common key



Merged Employee Info

```
pd.concat([df1, df2])  
pd.merge(employees, departments, on="Department_ID")
```

concat stacks datasets on top of each other; merge joins them side-by-side using a shared key column. Keep this brief if time is tight.

CODE

Exporting Cleaned Data

```
df.to_csv("cleaned_employee_data.csv", index=False)
```

- `index=False` stops Pandas from writing the row-number column into the file
- Analysis usually ends by producing a clean dataset for someone else — a colleague, a dashboard, or a model

07

GUIDED EDA CHALLENGE

Your turn — think first, then code

CHALLENGE

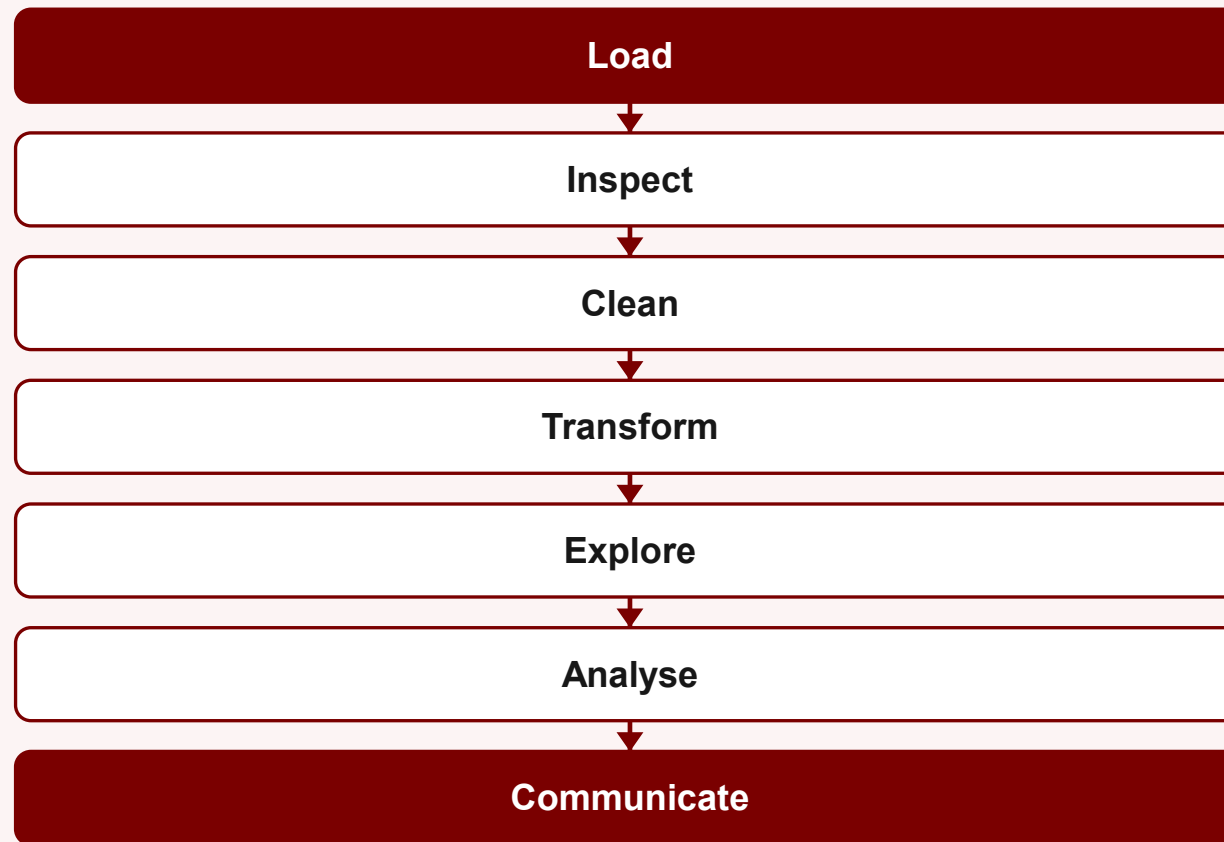
Explore the Dataset Yourself

- 1 How many employees are in the dataset?
- 2 How many columns are there?
- 3 Which columns contain missing values?
- 4 How many duplicate records exist?
- 5 What is the average age?
- 6 What is the median salary?
- 7 Which employees have the highest salaries?
- 8 What is the Active/Inactive status distribution?
- 9 Which department-region has the highest average salary?
- 10 What is the split between remote and non-remote employees?
- 11 What problems do you notice with Join_Date?
- 12 What problems do you notice with Salary?
- 13 What data-cleaning steps would you recommend?

What would you do first? Think before you type.

DIAGRAM

Mini EDA Workflow Recap



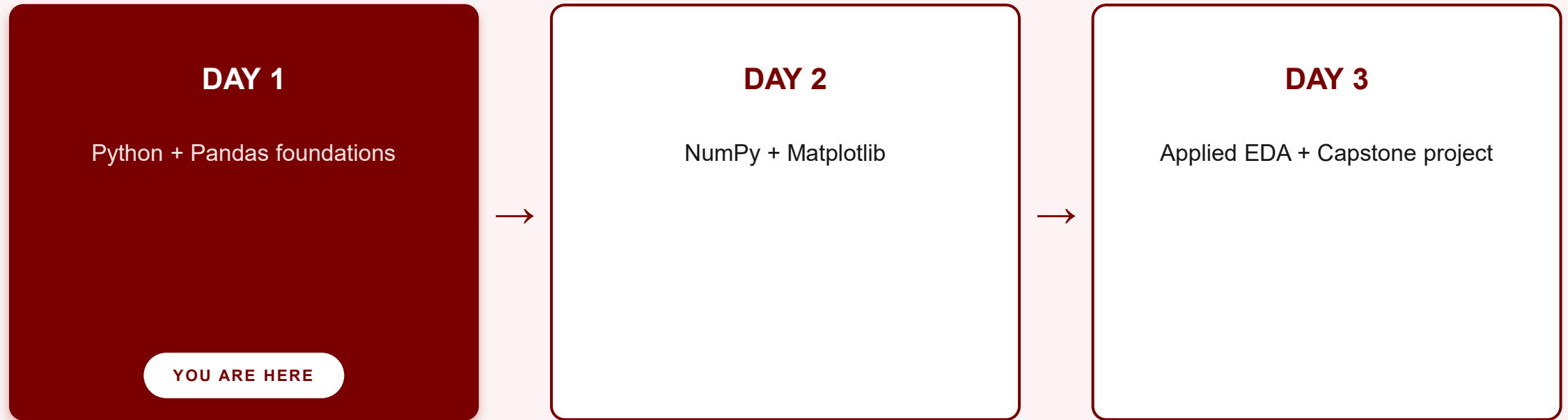
This is the loop you'll repeat on every new dataset you ever touch.

RECAP

Today We Learned

- ✓ What Data Science is
- ✓ Different data roles
- ✓ What EDA is
- ✓ Why EDA matters
- ✓ Python fundamentals for data analysis
- ✓ Pandas basics — Series & DataFrame
- ✓ How to inspect a dataset
- ✓ How to filter and analyse data
- ✓ How to identify data quality problems
- ✓ How to begin cleaning messy data
- ✓ How to use GroupBy for analysis

Where We Go From Here



**“Don't just clean the data.
Ask questions about it.”**

Thank you — see you for Day 2.

Abdou Salam Sisawo

Instructor · TensorOrbit EDA Bootcamp